

CSE-4301
Object Oriented Programming
2023-2024

Week-5

Operator Overloading

-Faisal Hussain

Assistant Professor, Department Of Computer Science And Engineering, IUT

Example of Operator Overloading

```
// add distance d1,d2 and store the result in d3  
d3.addDistance(d1,d2);
```

Example of Operator Overloading

```
// add distance d1,d2 and store the result in d3  
d3.addDistance(d1,d2);
```

```
//This is more readable  
//add distance d1,d2 and store the result in d3  
d3 = d1 + d2;
```

Example of Operator Overloading

```
// add distance d1,d2 and store the result in d3  
d3.addDistance(d1,d2);
```

```
//This is more readable  
//add distance d1,d2 and store the result in d3  
d3 = d1 + d2;
```

This requires new meaning of (+)
operator where the operands are
Distance Class object

Operator Overloading

- C++ includes a rich store of **45 operators**
(=, +, -, *, /, <, >, <=, >=, ++, -- so on...)
- These operators are **defined** automatically for the fundamental types (int, float, etc.).
- When we define a **class**, we are **creating a new type**.
- **Most of the C++ operators can be overloaded to apply to your new class type**

Operator Overloading: List of Operators

Operators that can be overloaded							
+	-	*	/	%	^	&	
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	=	<<	>>	>>=
<<=	==	!=	<=	>=	&&		++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Fig. 11.1 | Operators that can be overloaded.

Operators that cannot be overloaded			
.	.*	::	?:

Fig. 11.2 | Operators that cannot be overloaded.

Operator Overloading

- ❑ Operator overloading is **not automatic** process.
- ❑ Must write **operator-overloading functions** to perform the desired operations. Note that you can write anything in the function. A good programming practice will be to write such a way that the operation should be intuitive for the class.
- ❑ There are three ways to write an operator overloading function
 - ❑ **member function**
 - ❑ **non-member function**
 - ❑ **friend function**

Rules and Restrictions on Operator Overloading

Cannot Change

- Precedence
- Associativity
(how operators of the same precedence are grouped in the absence of parentheses)
- Arity
(the number of arguments or operands taken)

Cannot Create

- new operators
- meaning of how an operator works

See the reference for precedence and associativity of operators in C++ in
https://en.cppreference.com/w/cpp/language/operator_precedence

Operator Overloading (Unary Operator)

```
int main() {  
    Counter c1, c2;  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
  
    ++c1;                //increment c1  
    ++c2;                //increment c2  
    ++c2;                //increment c2  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count() << endl;  
    return 0;  
}
```

This increase the **count** variable of the object by 1.

c1=0
c2=0
c1=1
c2=2

Operator Overloading (Unary Operator)

```
class Counter
{
private:
    unsigned int count;
public:
    Counter() : count(0) { }
    unsigned int get_count() {
        return count;
    }
    void operator ++ () {
        ++count;
    }
};
```

Operator overloading function name will be the intended operator with the word **operator** as prefix.

Operator Overloading (Unary Operator)

```
int main() {  
    Counter c1, c2;  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
  
    ++c1;           //increment c1  
    ++c2;           //increment c2  
    ++c2;           //increment c2  
    c1++;           //postfix increment  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count() << endl;  
    return 0;  
}
```

c1=0
c2=0
c1=2
c2=2

Operator Overloading (Unary Operator)

```
int main() {  
    Counter c1, c2;  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
  
    ++c1;           //increment c1  
    ++c2;           //increment c2  
    ++c2;           //increment c2  
    c1++;           //postfix increment  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count() << endl;  
    return 0;  
}
```

This was **prefix** increment.

c1=0
c2=0
c1=2
c2=2

Operator Overloading (Unary Operator)

```
int main() {  
    Counter c1, c2;  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
  
    ++c1;           //increment c1  
    ++c2;           //increment c2  
    ++c2;           //increment c2  
    c1++;           //postfix increment  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count() << endl;  
    return 0;  
}
```

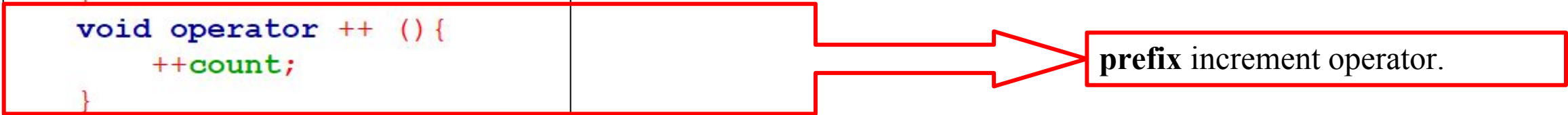
This was **prefix** increment.

This is postfix increment. This increase the **count** variable of the object by 1.

c1=0
c2=0
c1=2
c2=2

Operator Overloading (Unary Operator)

```
class Counter
{
private:
    unsigned int count;
public:
    Counter() : count(0) { }
    unsigned int get_count() {
        return count;
    }
    void operator ++ () {
        ++count;
    }
    void operator ++ (int) {
        ++count;
    }
};
```



prefix increment operator.

Operator Overloading (Unary Operator)

```
class Counter
{
private:
    unsigned int count;
public:
    Counter() : count(0) { }
    unsigned int get_count() {
        return count;
    }
    void operator ++ () {
        ++count;
    }
    void operator ++ (int) {
        ++count;
    }
};
```

postfix increment operator.

Operator Return Values

```
int main() {  
    Counter c1, c2;  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
  
    c2 = ++c1;  
    ///cout<< (++c1).get_count();  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count() << endl;  
    return 0;  
}
```

The operator++() has return type **void**. Need to change.

Operator Return Values

```
Counter Counter::operator ++ () {  
    ++count;  
    Counter temp;  
    temp.count = count;  
    return temp;  
}  
  
Counter Counter::operator ++ (int) {  
    Counter temp;  
    temp.count = count;  
    ++count;  
    return temp;  
}
```

Returning **Counter** Object.

In both case creating a temp object initialized with expected count value and return

Operator Return Values

```
Counter Counter::operator ++ () {  
    Counter temp;  
    temp.count = ++count;  
    return temp;  
}  
  
Counter Counter::operator ++ (int) {  
    Counter temp;  
    temp.count = count++;  
    return temp;  
}
```

An alternate option.

Operator Return Values

```
Counter::Counter(int val) {  
    count = val;  
}  
Counter Counter::operator ++ () {  
    Counter temp(++count);  
    return temp;  
}  
Counter Counter::operator ++ (int) {  
    Counter temp(count++);  
    return temp;  
}
```

Another alternate option.
Using one-argument constructor.

Operator Return Values

```
[-] Counter::Counter(int val) {  
    count = val;  
}  
[-] Counter Counter::operator ++ () {  
    return Counter(++count);  
}  
[-] Counter Counter::operator ++ (int) {  
    return Counter(count++);  
}
```

Another alternate option.

Creating an object without name -
nameless object.

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5), c3(10);  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    c3 = c1+c2;  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    return 0;  
}
```

Binary Operator (+), Both operand is Counter Object, expecting Counter Object as return type (following L.H.S. of assignment (=) operation).

Output

```
c1=0  
c2=5  
c3=10  
c1=15  
c2=5  
C3=10
```

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5), c3(10);  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    c3 = c1+c2;  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    return 0;  
}
```

Calling Object is the **left operand**

return_type operator + (**type** right_operand)

Output

c1=0
c2=5
c3=10
c1=15
c2=5
C3=10

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5), c3(10);  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    c3 = c1+c2;  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    return 0;  
}
```

```
Counter Counter::operator + (Counter c) {  
    return Counter(count+c.count);  
}
```

Calling Object is the **left operand**

return_type operator + (**type** right_operand)

Output

```
c1=0  
c2=5  
c3=10  
c1=15  
c2=5  
C3=10
```

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5), c3(10);  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    c3 = c1+c2;  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    return 0;  
}
```

`const` member function ensures that the member variable of the calling object will be unchanged.

```
Counter Counter::operator + (Counter c) const {  
    return Counter(count+c.count);  
}
```

Output

```
c1=0  
c2=5  
c3=10  
c1=15  
c2=5  
C3=10
```


Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5), c3(10);  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    c3 = c1+c2;  
  
    cout << "\nc1=" << c1.get_count();  
    cout << "\nc2=" << c2.get_count();  
    cout << "\nc3=" << c3.get_count();  
  
    return 0;  
}
```

`const` parameter ensures the object remain unchanged.

```
Counter Counter::operator + (const Counter& c) const {  
    return Counter(count+c.count);  
}
```

Output

```
c1=0  
c2=5  
c3=10  
c1=15  
c2=5  
C3=10
```

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5);  
    c1 = c2 + 10;  
}
```

right operand is `int`.

```
Counter Counter::operator+(int val) const {  
    return Counter(count + val);  
}
```

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5);  
    c1 = 10 + c2;  
}
```



left operand is int.
Member function approach is not possible.

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5);  
    c1 = 10 + c2;  
}
```

left operand is int.
Member function approach is not possible.

non-member function approach

```
Counter operator+(int val, const Counter& c) {  
    return Counter(c.get_count() + val);  
}
```

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5);  
    c1 = 10 + c2;  
}
```

left operand is int.
Member function approach is not possible.

non-member function approach

```
Counter operator+(int val, const Counter& c) {  
    return Counter(c.get_count() + val);  
}
```

First parameter is left operand, and Second parameter is right operand.
A non-member function cannot access private members.

Operator Overloading (Binary Operator)

```
int main() {  
    Counter c1, c2(5);  
    c1 = 10 + c2;  
}
```

left operand is int.
Member function approach is not possible.

non-member function approach

```
Counter operator+(int val, const Counter& c) {  
    return Counter(c.get_count() + val);  
}
```

First parameter is left operand, and Second parameter is right operand.
A non-member function cannot access private members.

All member function approach of operator overloading function can be converted to non-member function approach following these rules.

Operator Overloading (Binary Operator)

non-member friend function approach

A non-member function can be declared as **friend** to a class.
This allows the function to access the private members of the class.

Operator Overloading (Binary Operator)

non-member friend function approach

```
Counter operator+(int val, const Counter& c) {  
    return Counter(c.count + val);  
}
```

A non-member function can be declared as **friend** to a class.
This allows the function to access the private members of the class.

Operator Overloading (Binary Operator)

non-member friend function approach

```
class Counter{  
    // Partial class definition.  
    friend Counter operator+(int, const Counter&);  
};
```

```
Counter operator+(int val, const Counter& c){  
    return Counter(c.count + val);  
}
```

A non-member function can be declared as **friend** to a class.
This allows the function to access the private members of the class.

Operator Overloading (Binary Operator)

non-member friend function approach

```
class Counter{  
    // Partial class definition.  
    friend Counter operator+(int, const Counter&);  
};
```

```
Counter operator+(int val, const Counter& c){  
    return Counter(c.count + val);  
}
```

`operator +` function declared as `friend` to Counter class.

A non-member function can be declared as `friend` to a class.
This allows the function to access the private members of the class.

Operator Overloading (Binary Operator)

non-member friend function approach

```
class Counter{  
    // Partial class definition.  
    friend Counter operator+(int, const Counter&);  
};
```

`operator +` function declared as `friend` to Counter class.

```
Counter operator+(int val, const Counter& c){  
    return Counter(c.count + val);  
}
```

`c.count` is private and can be accessed in the friend function

A non-member function can be declared as `friend` to a class.
This allows the function to access the private members of the class.

More about friend function in future.

Operator Overloading (Binary Operator)

member function declaration
Relational operators

```
bool operator <  (const Counter& right) const;  
bool operator >  (const Counter& right) const;  
bool operator <= (const Counter& right) const;  
bool operator >= (const Counter& right) const;  
bool operator == (const Counter& right) const;  
bool operator != (const Counter& right) const;
```

Operator Overloading (Type Conversion)

Conversion from User Defined to Basic type

Operator Overloading (Type Conversion)

Conversion from User Defined to Basic type

Converting a Counter class to int type.

Operator Overloading (Type Conversion)

Conversion from User Defined to Basic type

Converting a Counter class to int type.

```
int main() {  
    Counter c1(5);  
    int val;  
    val = static_cast<int>(c1);  
}
```

Operator Overloading (Type Conversion)

Conversion from User Defined to Basic type

Converting a Counter class to int type.

```
int main() {  
    Counter c1(5);  
    int val;  
    val = static_cast<int>(c1);  
}
```

Converting c1 to int

Operator Overloading (Type Conversion)

Conversion from User Defined to Basic type

Converting a Counter class to int type.

```
int main() {  
    Counter c1(5);  
    int val;  
    val = static_cast<int>(c1);  
}
```

Converting c1 to `int`

```
Counter::operator int() {  
    return count;  
}
```

operator `int()` does not require any return type mentioned. It must return `int`.

Operator Overloading (Type Conversion)

Conversion from Basic type to User defined

Converting int to Counter class.

```
int main() {  
    Counter c2;  
    c2 = 10;  
    cout<< c2.getCount()<<endl;  
}
```

Operator Overloading (Type Conversion)

Conversion from Basic type to User defined

Converting int to Counter class.

```
int main() {  
    Counter c2;  
    c2 = 10;  
    cout<< c2.getCount()<<endl;  
}
```

L.H.S. = R.H.S

This operation tries to convert R.H.S to the equivalent to the type of L.H.S.

Operator Overloading (Type Conversion)

Conversion from Basic type to User defined

Converting int to Counter class.

```
int main() {  
    Counter c2;  
    c2 = 10;  
    cout<< c2.getCount()<<endl;  
}
```

L.H.S. = R.H.S

This operation tries to convert R.H.S to the equivalent to the type of L.H.S.

Converting `int` to `c2`

Operator Overloading (Type Conversion)

Conversion from Basic type to User defined

Converting int to Counter class.

```
int main() {  
    Counter c2;  
    c2 = 10;  
    cout<< c2.getCount()<<endl;  
}
```

```
Counter::Counter(int val)  
:count(val) {  
}
```

L.H.S. = R.H.S

This operation tries to convert R.H.S to the equivalent to the type of L.H.S.

Converting `int` to `c2`

One argument constructor.

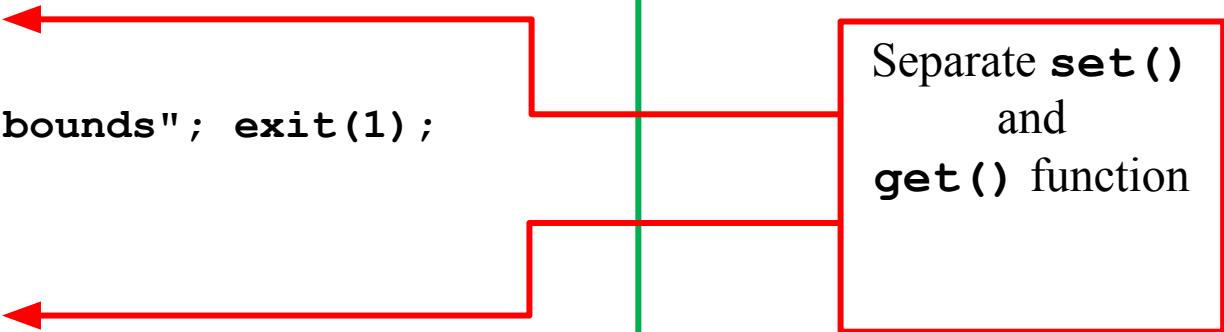
Operator Overloading (Subscript Operator ([]))

```
const int LIMIT = 100;
class safearray{
private:
    int arr[LIMIT];
public:
    void putel(int n, int elvalue){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds"; exit(1);
        }
        arr[n] = elvalue;
    }
    int getel(int n) const {
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};
```

Operator Overloading (Subscript Operator ([]))

```
const int LIMIT = 100;
class safearray{
private:
    int arr[LIMIT];
public:
    void putel(int n, int elvalue){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds"; exit(1);
        }
        arr[n] = elvalue;
    }
    int getel(int n) const {
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};
```

Separate **set()**
and
get() function



Operator Overloading (Subscript Operator ([]))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& access(int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds"; exit(1);
        }
        return arr[n];
    };
};
```


Operator Overloading (Subscript Operator ([]))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& access(int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds"; exit(1);
        }
        return arr[n];
    };
};
```

Single **access()**
function using **return** by
reference

Operator Overloading (Subscript Operator ([]))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& operator [] (int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};
```

Operator Overloading (Subscript Operator (`[]`))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& operator [] (int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};
```

Overloaded subscript
operator (`[]`) using `return`
reference

Operator Overloading (Subscript Operator (`[]`))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& operator [] (int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};

int main(){
    safearray sa1;
    for(int j=0; j<LIMIT; j++)
        sa1[j] = j*10;
    for(j=0; j<LIMIT; j++) {
        int temp = sa1[j];
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}
```

Overloaded subscript operator (`[]`) using `return` reference

Operator Overloading (Subscript Operator (`[]`)

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& operator [] (int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};

int main(){
    safearray sa1;
    for(int j=0; j<LIMIT; j++)
        sa1[j] = j*10;
    for(j=0; j<LIMIT; j++) {
        int temp = sa1[j];
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}
```

Overloaded subscript operator (`[]`) using `return` reference

Operator Overloading (Subscript Operator (`[]`))

```
class safearray{
private:
    int arr[LIMIT];
public:
    int& operator [] (int n){
        if( n< 0 || n>=LIMIT ){
            cout << "\nIndex out of bounds";exit(1);
        }
        return arr[n];
    }
};

int main(){
    safearray sa1;
    for(int j=0; j<LIMIT; j++)
        sa1[j] = j*10;
    for(j=0; j<LIMIT; j++) {
        int temp = sa1[j];
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}
```

Overloaded subscript operator (`[]`) using `return` reference

Operator Overloading (Type Conversion)

Conversion from User defined to User defined

```
class time12{
private:
    bool pm;
    int hrs;
    int mins;
public:
    time12() : pm(true), hrs(0), mins(0){ }
    time12(bool ap, int h, int m)
        : pm(ap), hrs(h), mins(m){ }
    void display() const {
        cout << hrs << ':';
        if(mins < 10)
            cout << '0';
        cout << mins << ' ';
        string am_pm = pm ? "p.m." : "a.m.";
        cout << am_pm;
    }
};
```

Operator Overloading (Type Conversion)

Conversion from User defined to User defined

```
class time24{
private:
    int hours;
    int minutes;
    int seconds;
public:
    time24() : hours(0), minutes(0), seconds(0) {
    }

    time24(int h, int m, int s)
        :hours(h), minutes(m), seconds(s){    }
    void display() const {
        if(hours < 10)    cout << '0';
        cout << hours << ':';
        if(minutes < 10)  cout << '0';
        cout << minutes << ':';
        if(seconds < 10)  cout << '0';
        cout << seconds;

    }
    operator time12() const;
```

Function in source object

```
time12 t12 = t24;
```

This statement will call the time12() function. As L.H.S. is time12 object so automatic type conversion will be imposed on R.H.S.

Operator Overloading (Type Conversion)

Conversion from User defined to User defined

```
time24::operator time12() const {  
    int hrs24 = hours;  
    bool pm = hours < 12 ? false : true;  
    int roundMins = seconds < 30 ? minutes : minutes+1;  
    if(roundMins == 60) {  
        roundMins=0;  
        ++hrs24;  
        if(hrs24 == 12 || hrs24 == 24)  
            pm = (pm==true) ? false : true;  
    }  
    int hrs12 = (hrs24 < 13) ? hrs24 : hrs24-12;  
    if(hrs12==0) {  
        hrs12=12;  
        pm=false;  
    }  
    return time12(pm, hrs12, roundMins);  
}
```

Function in source object.
type operator of
destination object.

Operator Overloading (Type Conversion)

Conversion from User defined to User defined

```
time12::time12( time24 t24 ) {  
    int hrs24 = t24.getHrs  
    pm = t24.getHrs() < 12 ? false : true;  
    mins = (t24.getSecs() < 30) ?  
           t24.getMins() : t24.getMins()+1;  
    if(mins == 60) {  
        mins=0;  
        ++hrs24;  
        if(hrs24 == 12 || hrs24 == 24)  
            pm = (pm==true) ? false : true;  
    }  
    hrs = (hrs24 < 13) ? hrs24 : hrs24-12;  
    if(hrs==0) {  
        hrs=12;  
        pm=false;  
    }  
}
```

Function in destination
object.

Constructor of destination
class with source class as
argument.